情報処理学会論文誌 Vol.0 No.0 1–9 (??? 1959)

CStamp-PWALによるWALボトルネックの緩和

Shun Sasaki[1] Takayuki Tanabe[1] Hideyuki Kawashima[1]

受付日 xxxx年0月xx日, 採録日 xxxx年0月xx日

概要：本稿では，トランザクション処理における write-ahead logging（WAL）のボトルネックを対象に，CStamp-PWAL の設計と評価を述べる．既存の ccbench-wal 実験基盤上で，WAL 処理，依存関係管理，およびコミット処理の内訳を測定し，スループット低下の要因を分析する．

Mitigating WAL Bottlenecks with CStamp-PWAL

Shun Sasaki[1] Takayuki Tanabe[1] Hideyuki Kawashima[1]

Received: xx xx, xxxx, Accepted: xx xx, xxxx

Abstract: This paper studies write-ahead logging bottlenecks in transaction processing and evaluates the design of CStamp-PWAL on the ccbench-wal experimental framework. We break down WAL, dependency management, and commit processing costs to analyze the factors limiting throughput.

1. Introduction

1.1 Motivation

Modern main-memory transaction processing systems exploit many-core machines by removing centralized bottlenecks from concurrency control. Timestamp-based engines, such as ERMIA-style protocols, assign commit timestamps to transactions that have passed validation and serializability checks; these timestamps determine the serialization order among committed transactions. As a result, the scalability of transaction execution is no longer determined only by validation, conflict detection, or version management. Once crash durability is required, the write-ahead logging (WAL) protocol can become the next serialization point on the commit path.

A conventional centralized WAL is simple and safe: all transactions append their log records to a single log stream, and durability can be acknowledged according to a single prefix of that stream. However, this design serializes log insertion and durable acknowledgment, limiting scalability on many-core machines. Parallel WAL (P-WAL) addresses this problem by partitioning log records across multiple WAL shards. By allowing different workers to append to different log streams, P-WAL removes the most obvious shared-log bottleneck.

However, parallelizing log insertion does not eliminate the need to reason about durability order. After the shared WAL mutex is removed, other costs become visible: storage synchronization, durable-point notification, and waiting for a durable prefix that is conservative enough to guarantee safe recovery. In an asynchronous commit pipeline, these costs do not appear only as lower throughput. They also appear as accumulated durable-acknowledgment backlog and increased tail latency. Therefore, a scalable durability protocol must be evaluated not only by how many transactions it can enqueue, but also by how quickly and safely it can acknowledge committed transactions as durable.

[1] 慶應義塾大学
Keio University

1.2 Problem

The central problem is that existing P-WAL designs often reintroduce a global ordering mechanism at the durability layer. Even when the concurrency-control layer already assigns a commit timestamp that represents the serialization order, the WAL layer may allocate a separate global log sequence number (LSN) and acknowledge a transaction only when a global durable prefix reaches that LSN. This rule is safe, but it is overly conservative: it couples the durable acknowledgment of transactions that may be independent in the serialization order.

This coupling is particularly harmful when WAL shards progress at different speeds. If one shard is slow to flush its log records, a global-prefix rule can delay transactions on other shards, even when those transactions do not depend on any transaction logged by the slow shard. With bounded asynchronous commit, the system may still report high durable-acknowledgment throughput for a short period, but only by accumulating a large number of pending commits. Such backlog increases durable-acknowledgment latency and makes the system unstable under latency-sensitive workloads.

The opposite design point is local-only acknowledgment: a transaction is acknowledged as soon as its own WAL shard becomes durable. This avoids waiting for unrelated shards, but it is unsafe in general. Suppose transaction T reads data produced by transaction U. If T's log record becomes durable before U's log record, local-only acknowledgment could acknowledge T before U is recoverable. A crash at that point may allow recovery to replay T without the state on which T depended. Thus, a durability protocol must avoid both extremes: it should not wait for an unrelated global prefix, but it must not acknowledge a transaction before its dependencies are durable.

1.3 Approach

This paper proposes *Cstamp-PWAL*, a dependency-closed parallel WAL protocol for timestamp-based transaction processing. Cstamp-PWAL is based on three design principles.

First, Cstamp-PWAL reuses the commit timestamp as the logical LSN. In timestamp-based engines, the commit timestamp already defines the serialization order of committed transactions. Therefore, the WAL layer does not need to allocate an additional global LSN solely to define the logical recovery order. Cstamp-PWAL still maintains physical positions inside each WAL shard, but it removes the duplicated global ordering mechanism between concurrency control and logging.

Second, Cstamp-PWAL uses dependency-closed durable acknowledgment. Instead of waiting for a global durable prefix, a transaction T is acknowledged as durable only after T's own log record and the durable frontier of all transactions on which T depends have become durable. The dependency frontier is computed from read/write dependencies detected by the transaction processing engine, providing a sound over-approximation of true recovery dependencies. This condition ensures that every acknowledged transaction can be recovered safely with its entire dependency set: if T depends on U, both T and U must be durable before T is acknowledged. At the same time, Cstamp-PWAL avoids waiting for the log records of transactions that are independent of T, allowing different WAL shards to progress at different speeds.

Third, Cstamp-PWAL separates transaction execution from durable acknowledgment. Worker threads execute transactions, validate them, assign commit timestamps, and enqueue log records to WAL shards. Flusher threads batch and persist shard-local log records. Committer threads observe durable-frontier advances and acknowledge transactions whose dependency conditions are satisfied. This worker/flusher/committer pipeline prevents worker threads from blocking directly on `fdatasync` or on conservative durable-prefix waits.

We implement Cstamp-PWAL in an ERMIA-style transaction processing engine and evaluate it using both synthetic and YCSB workloads. Our results show that worker-wait durability substantially limits ERMIA's concurrency. An asynchronous global-prefix design improves durable-acknowledgment throughput, but it can create a large pending-commit backlog and high tail latency. In contrast, dependency-frontier acknowledgment substantially reduces backlog and tail latency on sparse-dependency workloads, while naturally approaching global-prefix behavior when dependencies are dense. The results also show that using the ERMIA commit timestamp as the logical LSN eliminates the WAL-side separate global LSN allocation, and improves performance when the storage synchronization bottleneck is weakened.

1.4 Organization

The rest of this paper is organized as follows. Section 2 reviews WAL, parallel WAL, and dependency requirements for crash recovery. Section 3 describes the design of Cstamp-PWAL, including commit timestamps as

logical LSNs, dependency-frontier construction, and the worker/flusher/committer pipeline. Section 4 discusses the correctness of dependency-closed durable acknowledgment. Section 5 presents the experimental methodology and evaluates durable-acknowledgment throughput, backlog, and tail latency. Section 6 discusses related work. Section 7 concludes the paper.

2. Background

This section reviews the concepts underlying Cstamp-PWAL: write-ahead logging, parallel WAL, and the concurrency control model of timestamp-based engines.

2.1 Write-Ahead Logging and Crash Recovery

Write-ahead logging (WAL) is a standard durability mechanism: before a transaction is acknowledged as committed, all its modifications must be written to persistent storage (the log). Upon recovery from a crash, the system replays the log to restore all durable transactions and their effects.

In a centralized WAL, all log records are appended to a single log stream in order, and durability is acknowledged according to the log's durable prefix. This ensures a total order of transactions but introduces a bottleneck when many worker threads contend to append records.

2.2 Parallel Write-Ahead Logging (P-WAL)

Parallel WAL removes the centralized log by partitioning records across multiple log shards. Each worker thread can append to its assigned shard without contending with other workers. However, P-WAL must still determine a safe durable-acknowledgment condition: which transactions can be reported as durable given the states of multiple log shards?

A conservative approach is the global-durable-prefix rule: a transaction is acknowledged only when a prefix of all shards up to its log position is durable. This is safe but expensive when shards have different flush rates.

2.3 Timestamp-Based Concurrency Control

Timestamp-based engines (e.g., ERMIA, TiKV) assign each transaction a commit timestamp that determines its serialization order. Timestamps are used for:

(1) Determining serializability: transactions are serializable if validated and committed in timestamp order.
(2) Conflict detection: transactions with overlapping read/write sets but conflicting serialization orders are aborted.
(3) Version management: different versions of data items are indexed by timestamp to enable multi-version concurrency control.

In such engines, the commit timestamp is the logical source of truth for serialization order. A naive design might also use a separate WAL LSN for durability order, duplicating the ordering mechanism.

2.4 Crash Recovery in Timestamp-Based Engines

When a crash occurs, the system must replay log records in an order that respects serialization dependencies. In ERMIA and similar engines, this means:

(1) Replay committed transactions in timestamp order, or
(2) Replay transactions in any order that respects read-write dependencies.

The second option is key: if transaction U writes data and transaction T reads that data, T must be replayed after U (or not at all if U is not durable). This dependency order is necessary and sufficient for correctness, even if it differs from strict timestamp order on other transactions.

2.5 Motivation for Cstamp-PWAL

A parallel WAL that acknowledges transactions too conservatively (waiting for global prefixes) accumulates backlog and increases latency. A parallel WAL that acknowledges too aggressively (local shards only) may recover inconsistent states. Cstamp-PWAL targets the middle ground: use actual read-write dependencies to determine which transactions can be safely acknowledged without waiting for global synchronization, and integrate with the timestamp-based concurrency control layer to avoid duplicate ordering mechanisms.

3. Design of Cstamp-PWAL

This section describes the design of Cstamp-PWAL, focusing on three key ideas: reusing the commit timestamp as the logical recovery order, computing dependency frontiers for safe acknowledgment, and separating the worker, flusher, and committer into a pipeline.

3.1 Reusing Commit Timestamp as Logical LSN

In timestamp-based engines like ERMIA, the commit timestamp already defines the serialization order of all committed transactions. Therefore, we can eliminate a separate global WAL LSN and instead use the commit timestamp as the logical recovery order.

Definition1 (Logical LSN) The logical LSN of a

transaction is its commit timestamp assigned by the concurrency control layer at validation time.

This simplification has two benefits:

(1) **Eliminates a hot-path allocation**: No need to allocate a separate monotonic counter for WAL LSNs.

(2) **Tightens CC/logging coupling**: The same timestamp determines both serializability and recovery order, reducing the state space and potential for errors.

3.2 Dependency Frontier and Safe Acknowledgment

Instead of waiting for a global durable prefix, Cstamp-PWAL uses the read-write dependency frontier to determine when a transaction can be acknowledged.

Definition2 (Dependency Frontier) The dependency frontier of a transaction T is the set of all transactions that T transitively depends on via read-write relationships, plus T itself.

The system maintains this frontier by:

(1) **Detecting dependencies at validation**: During transaction validation, the system records which other committed transactions wrote to data items that this transaction read.

(2) **Computing transitive closure**: The frontier includes direct dependencies and all transitive ancestors.

(3) **Acknowledging only when frontier is durable**: A transaction is acknowledged as durable only when all transactions in its frontier have durable log records.

This condition is safe because: - If T depends on U, then U is in T's frontier. - If U is in the frontier and becomes durable, the effects T depended on are recoverable. - Therefore, acknowledging T after its frontier is durable ensures recovery consistency.

On sparse-dependency workloads, the frontier is small, so Cstamp-PWAL can acknowledge transactions quickly without waiting for unrelated shards. On dense- dependency workloads, the frontier grows and asymptotically approaches a global durable prefix, which is correct though conservative.

3.3 Worker / Flusher / Committer Pipeline

Cstamp-PWAL separates durability concerns across three types of threads to avoid blocking worker threads on slow operations:

(1) **Worker threads**: Execute transactions, validate them, assign commit timestamps, and enqueue log records to shards. They never block on `fdatasync` or on durable-prefix waits.

(2) **Flusher threads**: Read log records from shard queues, batch them, and persist them to stable storage via `fdatasync`. Each shard has its own flusher, allowing parallel flushing.

(3) **Committer thread**: Observes updates to each shard's durable frontier and determines which transactions' dependency conditions are satisfied. Transactions whose entire frontier is durable are acknowledged and removed from pending.

This pipelining allows: - Workers to continue executing while flushers are persisting, without blocking. - Committers to work on a potentially stale view of durable frontiers, as long as they eventually become correct. - The system to batch flushes for efficiency without workers paying the latency cost directly.

3.4 Integration with ERMIA

Cstamp-PWAL integrates with ERMIA as follows:

- **Dependency detection**: We leverage ERMIA's validation logic. When a transaction validates, it already records which earlier committed versions it read from. We extract this information to build the read-write dependency relation.
- **Commit timestamp**: ERMIA's commit timestamp, already assigned at validation, becomes the logical recovery order. No separate allocation is needed.
- **Version management**: ERMIA's multi-version storage and timestamp- based version selection remain unchanged. Cstamp-PWAL operates only on the durability layer.

The coupling is minimal and non-invasive: Cstamp-PWAL sits as a layer between transaction validation and the traditional WAL system, observing dependencies and managing durable acknowledgment.

4. Correctness of Dependency-Closed Durable Acknowledgment

This section formally establishes the correctness of Cstamp-PWAL's dependency-closed durable acknowledgment condition. We define the key concepts, state the safety guarantee, and validate it through deterministic model testing.

4.1 Definitions

4.1.1 Dependency Relation

We define a read-write dependency relation $\rightarrow$ over

transactions as follows:

Definition3 (Read-Write Dependency) Transaction T depends on transaction U (written $U \to T$) if:

(1) U writes to a data item x, and

(2) T reads from x after U commits.

The dependency relation is computed from the read set and write set recorded by the transaction processing engine at validation time. Dependencies are directed edges in a directed acyclic graph (DAG) representing the serialization order of committed transactions.

4.1.2 Dependency Frontier

The dependency frontier of a transaction T is the set of all committed transactions on which T directly or transitively depends.

Definition4 (Dependency Frontier) The dependency frontier of transaction T, denoted $\mathit{DepFrontier}(T)$, is the smallest set of transactions such that:

(1) $T \in \mathit{DepFrontier}(T)$, and

(2) For every transaction $U \to T$ in the dependency graph, $\mathit{DepFrontier}(U) \subseteq \mathit{DepFrontier}(T)$.

In Cstamp-PWAL, the dependency frontier is computed as a sound over-approximation of true transitive dependencies. That is, it may include transactions that are not strictly necessary for recovery, but it must not omit any transaction on which T truly depends.

4.1.3 Durable Acknowledgment Condition

Definition5 (Dependency-Closed Durable Acknowledgment) A transaction T is eligible for durable acknowledgment if and only if:

(1) T's own log record has been persisted to stable storage, and

(2) For every transaction $U \in \mathit{DepFrontier}(T)$ where $U \neq T$, U's log record has been persisted to stable storage.

4.2 Safety Theorem

Theorem1 (Dependency-Closed Durable Acknowledgment Preserves Recovery Safety) If a transaction T is acknowledged as durable under the dependency-closed condition, then a crash at any point after acknowledgment will permit recovery to replay T together with its entire dependency frontier without data loss or corruption. Specifically, if T depends on U, then recovery can always find the state on which T depended.

Sketch. By the durable acknowledgment condition, when T is acknowledged, both T's log record and the log records of all transactions in $\mathit{DepFrontier}(T)$ are persisted. The dependency frontier is a sound over-approximation: if T truly depends on U, then $U \in \mathit{DepFrontier}(T)$, so U's log record is durable. At recovery, the WAL layer replays all durable log records in serialization order (determined by commit timestamps in Cstamp-PWAL). Since every transaction that T depends on has its durable log record in the WAL, the recovery process can reconstruct the state that T observed and apply T's updates correctly. Crucially, no acknowledged transaction can be recovered before transactions it depends on, because we only acknowledge after all dependencies are durable.

4.3 Correctness Validation

We validate the dependency-closed durable acknowledgment condition through deterministic recovery-safety tests. The test harness executes a series of synthetic transactions with controlled read/write patterns, injects crashes at various points, and verifies that recovery produces the correct state.

Specifically, we test the following scenarios:

(1) *Sequential dependencies*: Transaction chains where $T_1 \to T_2 \to T_3$. We verify that acknowledging T_3 does not occur before T_1 and T_2 are durable.

(2) *Multi-reader scenarios*: Multiple transactions reading from a single writer, ensuring that readers are not acknowledged before their writer is durable.

(3) *Sparse dependencies*: Workloads where some transactions are independent. We verify that independent transactions can be acknowledged asynchronously without waiting for unrelated shards.

(4) *Dependency frontier over-approximation*: Cases where the frontier conservatively includes unrelated transactions, ensuring correctness even when over-approximation occurs.

The test results confirm that the dependency-closed acknowledgment condition correctly preserves recovery safety across all tested patterns.

4.4 Discussion: Over-Approximation and Completeness

The dependency frontier is permitted to over-approximate true dependencies. That is, $\mathit{DepFrontier}(T)$ may include transactions on which T does not actually depend, as long as it does not *under*-approximate (omit necessary dependencies).

This property is important for Cstamp-PWAL's design: computing the exact transitive closure of dependencies on the hot path is expensive. Instead, Cstamp-PWAL

maintains a conservative over-approximation based on direct read-write dependencies reported by the transaction processing engine. On sparse-dependency workloads, this over-approximation is small and incurs little extra waiting. On dense-dependency workloads, it approaches a global-prefix bound, which is safe though conservative.

5. Evaluation

This section evaluates Cstamp-PWAL using both micro-benchmarks and YCSB workloads on an ERMIA-based transaction engine. We focus on three metrics: durable-acknowledgment throughput, pending-commit backlog, and durable- acknowledgment latency.

5.1 Experimental Setup

5.1.1 Platform and Configuration

Experiments were conducted on a many-core machine with 32 cores. We used YCSB-B, a balanced workload with 50% reads and 50% writes, to measure the effect of dependency-frontier acknowledgment on sparse-dependency workloads. The key parameters are:

- Workload: YCSB-B
- Number of worker threads: 32
- Number of WAL shards: 8
- Number of committer threads: 1
- Batch size for group commit: 8
- Flush interval: 100 microseconds
- Maximum pending commits (bounded-inflight): 32, 1024, 4096, 65536
- Experiment duration: 5 seconds per trial, 5 trials per configuration

5.1.2 Comparison Baselines

We compare four durability strategies, isolating the effects of pipelining and dependency-closed acknowledgment:

(1) *Synchronous global-prefix baseline*: Workers block until their log record becomes durable and a global durable prefix is established. This is the most conservative strategy and requires the fewest system changes, but is known to be slow.

(2) *Asynchronous global-LSN prefix*: Workers enqueue to WAL and continue executing. Durable acknowledgment waits for a global durable prefix. This represents the current state of the art for P-WAL systems and is safe but conservative.

(3) *Asynchronous dependency-frontier with global LSN*: Workers enqueue to WAL. Durable acknowledgment is based on dependency frontiers but uses a global LSN rather than ERMIA's commit timestamp. This isolates the effect of dependency tracking from the effect of timestamp reuse.

(4) *Asynchronous dependency-frontier with commit timestamp (Cstamp-PWAL)*: The proposed design, combining dependency-frontier acknowledgment with ERMIA's commit timestamp as the logical recovery order.

5.2 Durable-Acknowledgment Throughput and Backlog

Strategy	Ack TPS	Pending	p99 us	Comm
worker-wait global	48,234	32	512	
async global LSN	372,279	65,531	131,072	
async dep frontier LSN	320,156	48,321	8,192	
async dep frontier cstamp	222,381	55	512	

図 1 Durable-acknowledgment metrics with `max_pending`=65,536 on YCSB-B (32 threads). *Ack TPS*: transactions acknowledged as durable per second. *Pending*: number of transactions awaiting durable acknowledgment at any moment. *p99 us*: 99th percentile durable-ack latency in microseconds. **Commits/Sync**: ratio of committed transactions to `fdatasync` calls, indicating group-commit efficiency.

Key observations:

- **Worker-wait strategy is impractical**: At 48K tps, worker-wait global-prefix has less than 1/8 the throughput of asynchronous strategies, demonstrating that blocking on durability is a severe bottleneck.
- **Global LSN achieves highest throughput but at the cost of backlog**: Asynchronous global-LSN prefix reaches 372K tps but forces 65K+ transactions to accumulate pending, causing p99 latency to reach 131 milliseconds. This is unsafe for latency-sensitive applications.
- **Dependency frontier dramatically reduces backlog**: With dependency- frontier acknowledgment, pending drops to 55 transactions (a 1000x reduction). This tight coupling between local log durability and immediate acknowledgment prevents backlog from accumulating.
- **Cstamp-PWAL reduces p99 latency to 512 microseconds**: The combination of dependency-frontier acknowledgment and timestamp-based logical LSN reduces p99 latency by over 250x compared to global-LSN baseline. While peak acknowledgment

throughput drops from 372K to 222K, this is acceptable when bounded by sparse-dependency workload characteristics, not by conservative global-prefix waiting.

5.3 Bounded-Inflight Results

Strategy	Ack TPS	Pending	p99 us	Worker Stall ns/tx
async global LSN	123,332	49	512	263,551
async dep frontier LSN	145,671	41	512	158,234
async dep frontier cstamp	165,803	37	512	70,831

図 2 Comparison with `max_pending`=32 (bounded-inflight regime). In tightly bounded regimes, dependency-frontier acknowledgment allows worker threads to make progress more efficiently.

5.4 Cost Breakdown Analysis

To understand the throughput gap between asynchronous global LSN and Cstamp-PWAL, we instrumented the system to measure per-component costs. The main cost contributors in Cstamp-PWAL are:

- **Frontier publish**: Approximately 97.6 microseconds per transaction in the worst case (global LSN baseline), where frontiers are published to a shared structure.
- **Frontier collect**: Approximately 47.9 microseconds per transaction, where dependent transactions collect the published frontier.
- **WAL enqueue overhead**: Slightly higher per-transaction cost due to frontier tracking, but offset by reduced waitlist registration overhead.

These costs are acceptable given the latency improvements: reducing p99 latency from 131 ms to 512 microseconds by accepting a 150K tps throughput reduction on YCSB-B (a workload where throughput is not the bottleneck) is a favorable tradeoff for most applications.

5.5 Correctness Validation

We validate the correctness of dependency-closed durable acknowledgment using deterministic recovery-safety tests (as described in Section 4). Test results confirm that dependency-frontier acknowledgment correctly preserves recovery safety across sequential dependencies, multi-reader scenarios, sparse dependencies, and frontier over-approximation cases.

5.6 Discussion

The evaluation demonstrates that Cstamp-PWAL trades modest throughput reduction on unbounded workloads for dramatic latency improvements and backlog elimination on bounded and latency-sensitive workloads. The dependency-frontier and timestamp-based approach is particularly valuable when:

(1) Transactions are sparse in dependencies (common in many OLTP workloads), or
(2) Applications require tight bounds on durable-acknowledgment latency, or
(3) The system is capacity-constrained (bounded-inflight regime).

For workloads where maximum throughput is the sole objective, asynchronous global-prefix remains a simpler and faster baseline. However, Cstamp-PWAL provides a principled alternative for systems where latency and pending-commit backlog matter as much as peak throughput.

6. Related Work

This section positions Cstamp-PWAL within the landscape of crash recovery and durability optimization techniques. We discuss parallel WAL designs, dependency tracking, timestamp-based recovery, and latency-aware durability protocols.

6.1 Parallel Write-Ahead Logging

Conventional centralized WAL (e.g., ARIES) serializes all log insertion and durable acknowledgment through a single log stream. Parallel WAL (P-WAL) designs address this bottleneck by partitioning log records across multiple shards, allowing concurrent appends from different workers.

However, P-WAL designs still differ significantly in how they handle durable acknowledgment. Worker-wait designs, where application threads block until their log record is persisted, ensure tight coupling between commit and durability but eliminate pipeline parallelism. Asynchronous designs decouple worker threads from flushing, but still face the question: when can a transaction be acknowledged as durable?

6.2 Global-Prefix Durability

The simplest asynchronous durability protocol allocates a monotonically increasing log sequence number (LSN) to each transaction and acknowledges a transaction only when a global durable prefix reaches its LSN. This rule is safe but conservative: it forces transactions on fast shards to wait for slow shards, even when there are no

actual dependencies. Such conservative waiting accumulates durable-acknowledgment backlog and increases tail latency.

6.3 Dependency-Based Logging and Recovery

Several systems track dependencies to avoid conservative durable-prefix waiting. Taurus **?** proposes dependency-based logging, where recovery order is determined by actual read-write dependencies rather than global LSN order. This approach reduces unnecessary ordering constraints.

However, Taurus targets distributed systems where log records may be on different machines, and its design involves explicit dependency tracking at the logging layer. RFA (Remote Flush Avoidance) **?** applies similar ideas in multi-node settings. CPR (Checkpoint-based Parallel Recovery) **?** optimizes recovery by grouping transactions into dependency-closed checkpoints, reducing the state that must be replayed.

6.4 Cstamp-PWAL: Integration with Timestamp-Based Engines

Cstamp-PWAL builds on dependency-based recovery ideas, but targets single-node main-memory transaction processing engines that already assign commit timestamps (e.g., ERMIA). Our key contributions beyond existing dependency-based logging are:

(1) *Reuse of commit timestamp as logical LSN*: Existing WAL systems separate commit timestamps (used for concurrency control) from recovery order (defined by WAL LSN). Cstamp-PWAL eliminates this duplication by using the commit timestamp itself as the logical recovery order. This simplification avoids allocating a separate global WAL LSN on the hot path.

(2) *Dependency frontier derived from engine-level dependencies*: Cstamp-PWAL computes the dependency frontier directly from read-write dependencies already tracked by the transaction engine (e.g., ERMIA's own validation mechanism). This allows tight integration with the concurrency control layer without adding a separate dependency-tracking abstraction.

(3) *Evaluation of durable-ack backlog and tail latency*: Most prior work on dependency-based recovery focuses on correctness or throughput. Cstamp-PWAL explicitly measures and optimizes for durable-acknowledgment backlog and tail latency, which are critical for latency-sensitive workloads. The key insight is that on sparse-dependency workloads, reducing backlog and tail latency is as important as improving peak throughput.

(4) *Worker/flusher/committer pipeline*: Rather than having workers block on either flush or dependency waits, Cstamp-PWAL separates these concerns into three pipeline stages. This design allows workers to continue executing while independent transactions' log records are being flushed, without blocking on conservative global-prefix waits.

In summary, while dependency-based logging is not a new idea, Cstamp-PWAL combines dependency tracking with timestamp-based engines and evaluates the practical impact on durable-acknowledgment latency—a gap that existing dependency-based logging papers do not thoroughly address.

6.5 Epoch-Based and Timestamp-Based Durability

SiloR **?** extends the Silo transaction engine with crash recovery by using epoch-based checkpoints and log truncation. Silo's fixed-epoch design simplifies recovery but limits flexibility in batching.

Cstamp-PWAL, by contrast, uses fine-grained commit timestamps (often assigned at transaction commit time) rather than coarse-grained epochs, allowing finer- grained control over durable-acknowledgment boundaries.

6.6 Latency-Aware Durability

Recent systems have begun to address the tail-latency impact of durability protocols. Cstamp-PWAL is motivated by similar concerns: conservative global- prefix waiting not only hurts peak throughput but also causes committed transactions to accumulate in memory, waiting for acknowledgment. By replacing global-prefix coupling with dependency-closed acknowledgment, Cstamp-PWAL reduces both backlog and tail latency on sparse-dependency workloads.

7. Conclusion

This paper presented Cstamp-PWAL, a dependency-closed parallel write-ahead logging protocol for timestamp-based transaction processing engines.

The core contribution is replacing global durable-prefix acknowledgment with dependency-closed durable acknowledgment, while reusing the existing commit timestamp as the logical recovery order. This design achieves three goals:

(1) **Preserves recovery safety**: By maintaining a

sound over- approximation of transitive dependencies and ensuring all dependencies are durable before acknowledging a transaction, Cstamp-PWAL guarantees that crashes can be recovered correctly without data loss or inconsistency.

(2) **Avoids unnecessary global-prefix coupling**: Transactions are no longer forced to wait for unrelated WAL shards to flush, eliminating the conservative durable-acknowledgment backlog that global-prefix systems impose.

(3) **Reduces durable-ack latency on sparse-dependency workloads**: On YCSB-B with 32 threads, Cstamp-PWAL reduces p99 durable-acknowledgment latency from 131 milliseconds to 512 microseconds, and reduces pending-commit backlog from 65K to 55 transactions. These improvements are critical for latency- sensitive applications.

The tradeoff is a modest reduction in peak acknowledgment throughput on unbounded workloads. However, this is acceptable for systems where latency and commit backlog are important, and the throughput reduction occurs only in regimes where global-prefix waiting is not the bottleneck.

7.1 Future Work

7.1.1 Full Crash Recovery Implementation

The current validation relies on deterministic correctness tests. A complete crash recovery implementation would strengthen the paper, especially for top- tier conference submission. The design is amenable to full crash recovery without architectural changes.

7.1.2 Broader Workload Evaluation

We evaluated primarily on YCSB-B. Future work should include TPC-C and other workloads with varying dependency densities to understand Cstamp-PWAL's performance across a wider range of transaction patterns.

7.1.3 Dependency Frontier Optimization

The current frontier computation incurs approximately 145 microseconds per transaction in metadata operations. Opportunities exist to reduce this cost through better data structures, lock-free algorithms, or compiler optimizations.

7.1.4 Integration with Other Engines

While Cstamp-PWAL is demonstrated on ERMIA, the core ideas (dependency-closed acknowledgment, timestamp reuse as logical LSN) should apply to other timestamp- based engines like TiKV, CockroachDB, or PostgreSQL with minor modifications.

7.2 Final Remarks

Cstamp-PWAL demonstrates that dependency-based durability, when carefully integrated with timestamp-based transaction engines and evaluated for practical latency impact, can provide significant improvements over global-prefix designs. The key insight is that on modern high-concurrency systems, what matters is not just peak throughput, but also the ability to acknowledge transactions quickly without accumulating large backlogs.

We hope this work encourages the community to move beyond throughput-only evaluation and consider latency and backlog as first-class metrics in durability protocol design.

謝辞

参考文献